Data 236- Distributed Systems

Homework-10

Kruthika Virupakshappa

Part 1- Customer Segmentation

1. Model Training Script (train_customer_model.py)

Create this script to perform the following:

- Simulate Data: Create 200 random data points for the two required features.
- Train Model: Fit a KMeans model with $k=5$ clusters to the data.
- Save: Save the trained model to customer_kmeans_model_yourname.pkl.
- Save Metadata: Save the feature names and the trained cluster centers to Customer_metadata.pkl.

train_customer_model.py

```
import numpy as np
import pickle
from sklearn.cluster import KMeans

np.random.seed(42)

annual_income = np.random.uniform(15, 137, 200)
spending_score = np.random.uniform(1, 99, 200)
X = np.column_stack([annual_income, spending_score])

model = KMeans(n_clusters=5, random_state=42, n_init=10)
model.fit(X)

with open("customer_kmeans_model_kruthika.pkl", "wb") as f:
   pickle.dump(model, f)

metadata = {
   "feature_names": ["Annual Income (k$)", "Spending Score (1-100)"],
   "cluster_centers": model.cluster_centers_,
   "X_train": X
}

with open("customer_metadata.pkl", "wb") as f:
   pickle.dump(metadata, f)

print("Model saved to customer_kmeans_model_kruthika.pkl")
print("Metadata saved to customer_metadata.pkl")
print(f"Cluster centers:\n{model.cluster_centers_}")
```

2. Streamlit Application

Create this script to perform the following:

- Load Resources: Use @st.cache_resource to load both .pkl files.

```
MODEL_FILE = "customer_kmeans_model_kruthika.pkl"
METADATA_FILE = "customer_metadata.pkl"

@st.cache_resource
def load_resources(model_file, metadata_file):
    if not os.path.exists(model_file) or not os.path.exists(metadata_file):
        st.error("Model files not found. Please run 'train_customer_model.py' first!")
        return None, None
    with open(model_file, "rb") as f:
        model = pickle.load(f)
    with open(metadata_file, "rb") as f:
        metadata = pickle.load(f)
    return model, metadata

model, metadata = load_resources(MODEL_FILE, METADATA_FILE)
```

- Input Widgets: Use two st.sidebar.slider widgets for the Annual Income and Spending Score features.

```
if model is not None:
    feature_names = metadata["feature_names"]
    X_train = metadata["X_train"]
    cluster_centers = metadata["cluster_centers"]

    st.sidebar.header("Customer Profile")
    annual_income = st.sidebar.slider(feature_names[0], 15.0, 137.0, 70.0, 1.0)
    spending_score = st.sidebar.slider(feature_names[1], 1.0, 99.0, 50.0, 1.0)
```

- Prediction: Use the loaded model to predict the Cluster ID for the user's input.

```
    input_features = np.array([[annual_income, spending_score]])
    predicted_cluster = model.predict(input_features)[0]

    st.metric(label="Predicted Cluster ID", value=int(predicted_cluster))
```

- Output:
    - Display the predicted Cluster ID using st.metric().

```
st.metric(label="Predicted Cluster ID", value=int(predicted_cluster))
```

    - Display a scatter plot of the original simulated data, clearly highlighting the predicted customer's location on the plot.

localhost:8501

Customer Profile

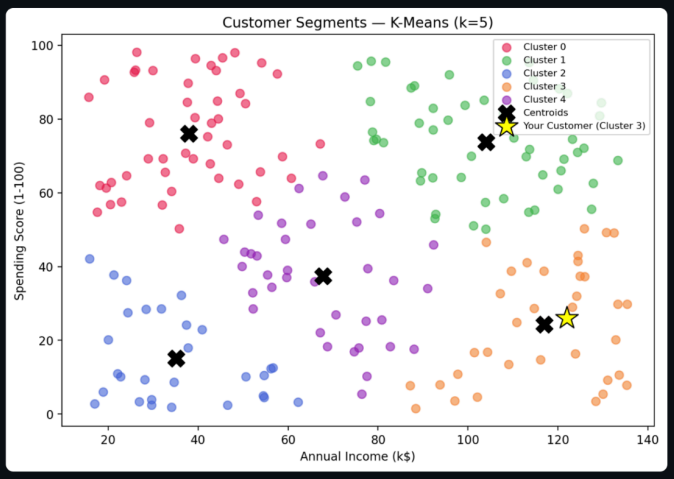
Annual Income (k$)

122.00

Spending Score (1-100)

26.00

3



Input Summary

Annual Income (k$)	Spending Score (1-100)	Assigned Cluster
122.0 k$	26.0	3

localhost:8501

Customer Profile

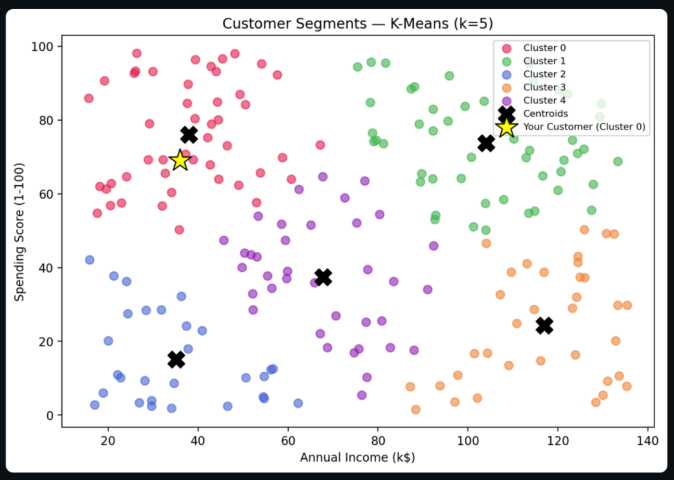
Annual Income (k$)

36.00

Spending Score (1-100)

69.00

0



Input Summary

Annual Income (k$)	Spending Score (1-100)	Assigned Cluster
36.0 k$	69.0	0

Part 2- Evaluating Your Kafka Agents with GEval

G-Eval: Consumes from the 'final' Kafka topic, which carries the full pipeline trace (plan, draft, final_answer) linked by correlation_id, then evaluates each stage with GEval using a local Ollama model as the judge.

Metrics

- Plan Quality- is the Planner's plan structured and actionable?
- Writer Helpfulness- does the Writer's draft directly answer the question?
- Reviewer Helpfulness- does the Reviewer's final answer improve on the draft?
- Final-vs-Draft- did the Reviewer meaningfully improve the draft?

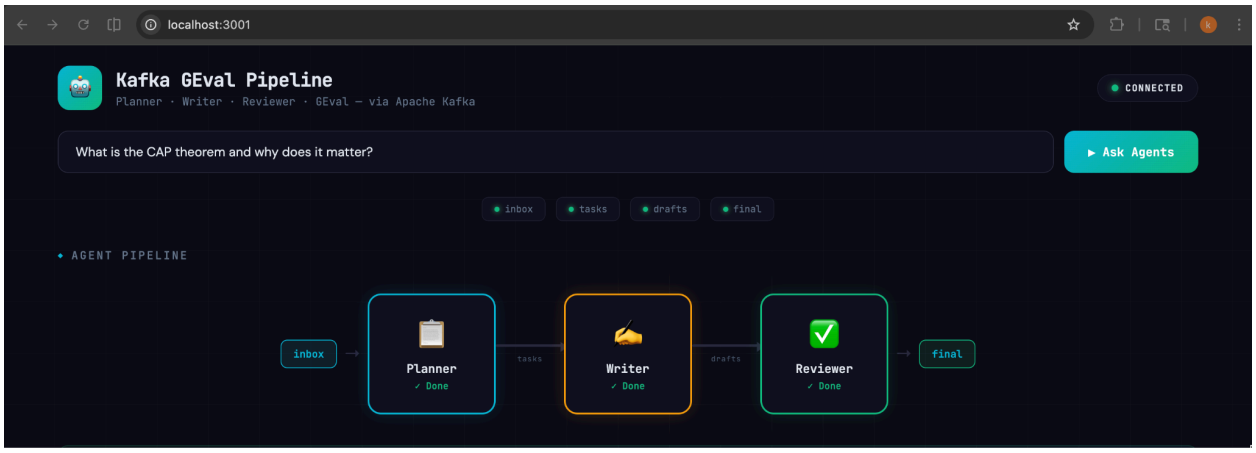
```
# OllamaJudge -- wraps local Ollama as GEval judge
class OllamaJudge(DeepEvalBaseLLM):
    def __init__(self, model: str = "llama3.2"):
        self._model = model

    def load_model(self): return self._model

    def generate(self, prompt: str, schema=None) -> str:
        kwargs = {"model": self._model,
                  "messages": [{"role": "user", "content": prompt}]}
        if schema is not None:
            kwargs["format"] = "json"   # force JSON for structured scoring
        return ollama.chat(**kwargs)["message"]["content"]

    async def a_generate(self, prompt: str, schema=None) -> str:
        return self.generate(prompt, schema=schema)

    def get_model_name(self) -> str:
        return f"ollama/{self._model}"
```

GEval metric definitions:

```
def build_metrics(judge: OllamaJudge) -> dict:
   plan_quality = GEval(
       name="Plan-Quality",
       criteria=(
           "Evaluate whether the ACTUAL_OUTPUT is a clear, numbered, actionable "
           "step-by-step plan that would genuinely help someone answer the INPUT "
           "question. Penalise vague, circular, or missing steps."
       ),
       evaluation_params=[
           LLMTestCaseParams.INPUT,
           LLMTestCaseParams.ACTUAL_OUTPUT,
       ],
       model=judge,
```

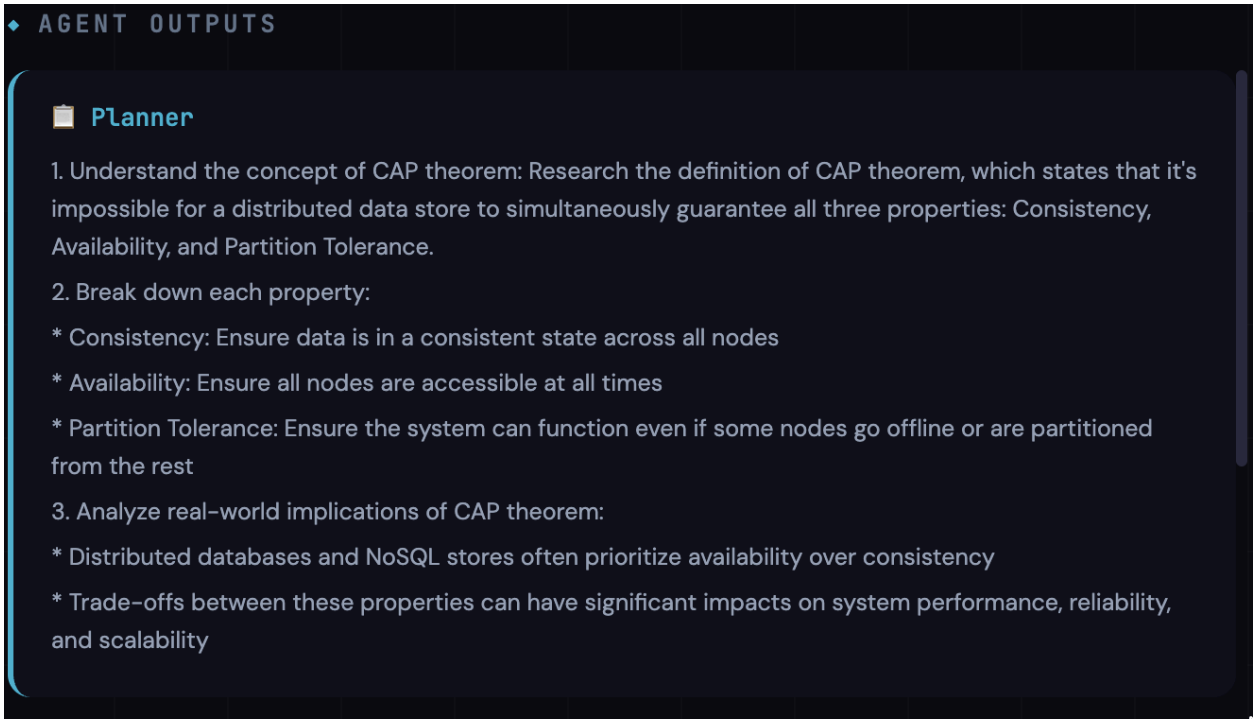
```
    )

    writer_helpfulness = GEval(
        name="Writer-Helpfulness",
        criteria=(
            "Determine whether the ACTUAL_OUTPUT directly and completely answers "
            "the INPUT question in clear, concise language. "
            "Penalise off-topic content, missing key facts, or unnecessary filler."
        ),
        evaluation_params=[
            LLMTestCaseParams.INPUT,
            LLMTestCaseParams.ACTUAL_OUTPUT,
        ],
        model=judge,
    )

    reviewer_helpfulness = GEval(
        name="Reviewer-Helpfulness",
        criteria=(
            "Determine whether the ACTUAL_OUTPUT (the Reviewer's final answer) "
            "directly and completely addresses the INPUT question. "
            "Score higher if it is accurate, clear, and well-organised."
        ),
        evaluation_params=[
            LLMTestCaseParams.INPUT,
            LLMTestCaseParams.ACTUAL_OUTPUT,
        ],
        model=judge,
    )

    draft_improvement = GEval(
        name="Final-vs-Draft-Improvement",
        evaluation_steps=[
            "EXPECTED_OUTPUT is the Writer's draft answer.",
            "ACTUAL_OUTPUT is the Reviewer's final answer.",
            "Check if the final answer is more accurate, clearer, or more complete "
            "than the draft.",
            "Score 1.0 if clearly improved, 0.5 if roughly the same quality, "
            "0.0 if worse.",
        ],
        evaluation_params=[
            LLMTestCaseParams.ACTUAL_OUTPUT,
            LLMTestCaseParams.EXPECTED_OUTPUT,
        ],
        model=judge,
    )

    return {
        "plan_quality": plan_quality,
        "writer_helpfulness": writer_helpfulness,
        "reviewer_helpfulness": reviewer_helpfulness,
        "draft_improvement": draft_improvement,
```

```
}
```

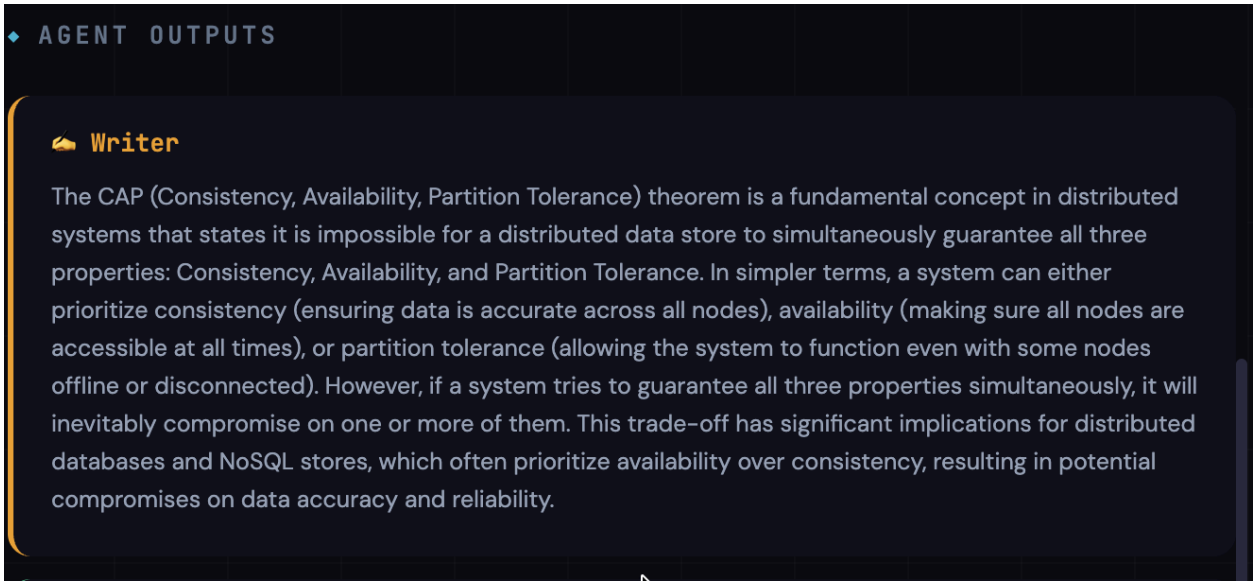

localhost:3001
Kafka GEval Pipeline
Planner · Writer · Reviewer · GEval — via Apache Kafka
CONNECTED
What is the CAP theorem and why does it matter?
▶ Ask Agents
inbox
tasks
drafts
final
AGENT PIPELINE
inbox
PLanner
✓ Done
tasks
Writer
✓ Done
drafts
Reviewer
✓ Done
final


KAFKA EVENT STREAM
16
11:37:28 PM THINKING planner #c1354e1f
11:37:34 PM ✓ COMPLETE planner #c1354e1f
11:37:34 PM KAFKA MSG → tasks #c1354e1f
11:37:34 PM ▶ START writer #c1354e1f
11:37:34 PM THINKING writer #c1354e1f
11:37:39 PM ✓ COMPLETE writer #c1354e1f
11:37:39 PM KAFKA MSG → drafts #c1354e1f
11:37:39 PM ▶ START reviewer #c1354e1f
11:37:39 PM THINKING reviewer #c1354e1f
11:37:44 PM ✓ COMPLETE reviewer #c1354e1f
11:37:44 PM KAFKA MSG → final #c1354e1f
11:37:44 PM DONE #c1354e1f
11:37:44 PM GEVAL START #c1354e1f
11:38:07 PM GEVAL DONE #c1354e1f

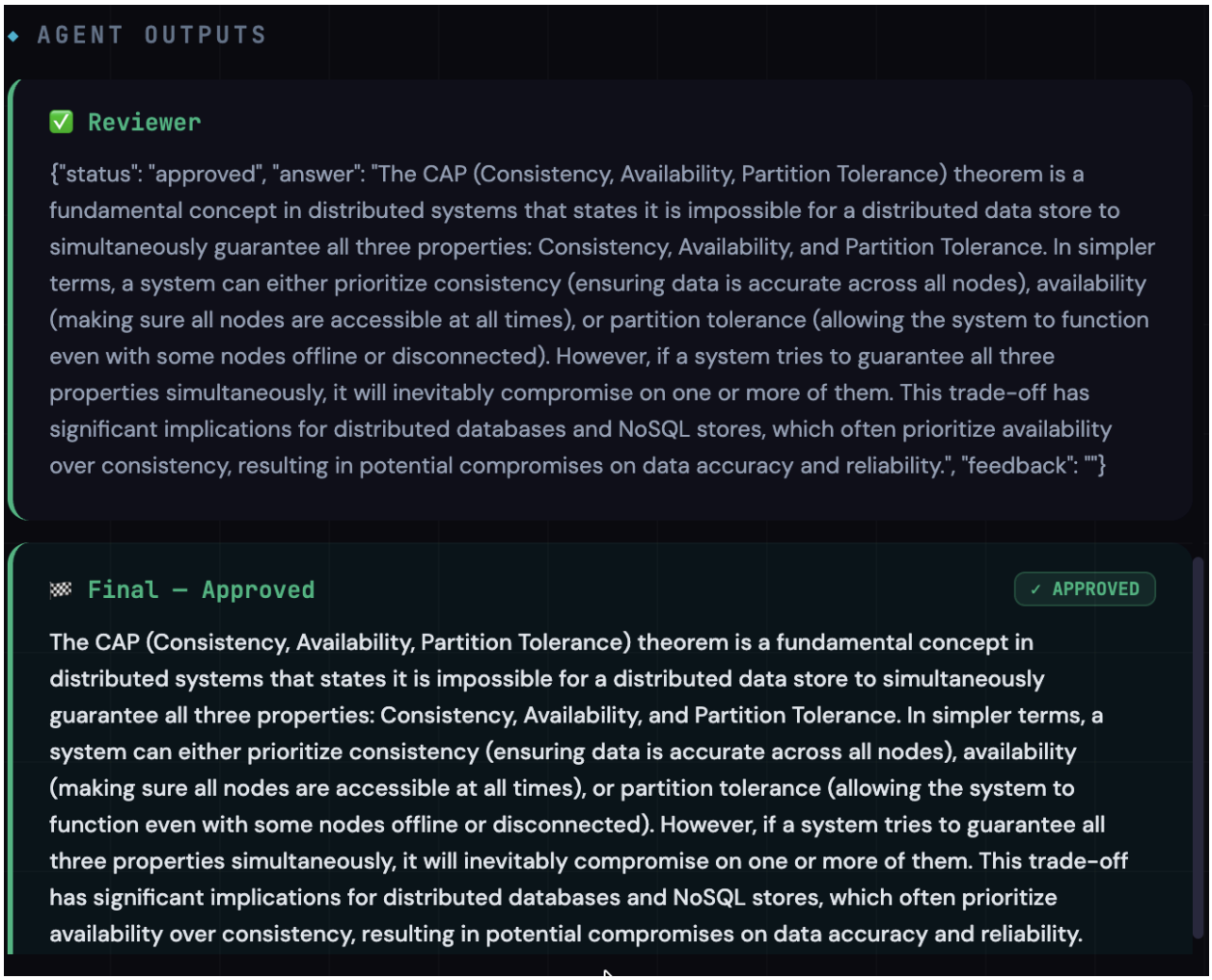
Planner:



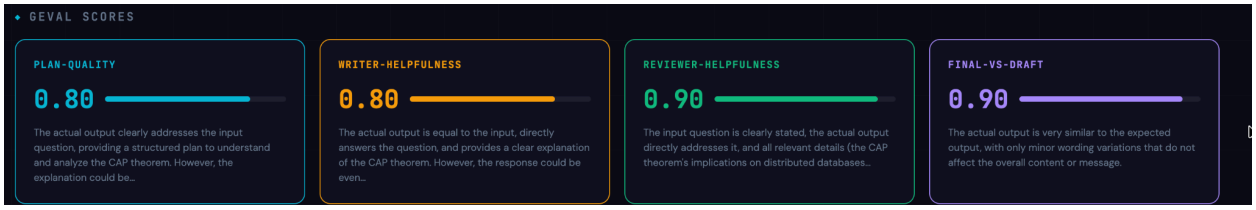
Writer:



Reviewer and approver:



GEval Scores:



In a multi-agent Kafka pipeline where Planner, Writer, and Reviewer operate asynchronously across topics, it is difficult to manually inspect whether each agent is contributing meaningfully. GEval addresses this by using a local LLM judge (llama3.2 via Ollama) to score each stage independently, evaluating plan structure, answer helpfulness, and whether the Reviewer genuinely improved the Writer's draft.

The scores revealed a clear improvement pattern across the pipeline: the Planner and Writer scored 0.80, indicating solid but improvable outputs, while the Reviewer pushed quality to

0.90, demonstrating that the review stage meaningfully refined the draft. The Final-vs-Draft-Improvement score of 0.90 confirms the Reviewer added real value rather than simply passing the draft through unchanged. This per-stage visibility is impossible with end-to-end testing alone. By linking traces via correlation_id, GEval pinpoints exactly which agent in the chain is underperforming, enabling targeted prompt improvements. Automated evaluation thus shifts agent development from guesswork to data-driven iteration, making coordination failures visible and fixable.